

FinSight FastAPI

Dokumentasi Teknis: Modul Scheduler & ML

DBS Coding Camp Capstone — 2026

Daftar Isi

1. Gambaran Umum	3
1.1. Struktur Direktori	3
1.2. Alur Data End-to-End	3
2. Modul ML (src/ml/)	4
2.1. Model Loader (model_loader.py)	4
2.1.1. Fungsi	4
2.1.2. Model Artifacts	4
2.2. NLP Service (nlp_service.py)	6
2.2.1. Konstanta	6
2.2.2. Fungsi	6
2.2.3. Alur Proses	7
2.3. Autoencoder Service (autoencoder_service.py)	8
2.3.1. Konstanta	8
2.3.2. Fungsi: preprocess_for_autoencoder(df, meta) -> np.ndarray	8
2.3.3. Fungsi: detect_anomalies(df, X_scaled, model_meta, df_baseline, weekly_report_id)	8
2.3.4. Fungsi: _build_anomaly_context(row, df_baseline) -> str	9
2.3.5. Domain Entity: DetectedAnomaly	9
2.4. Clustering Service (clustering_service.py)	10
2.4.1. Fungsi: compute_monthly_features(df_debit, df_all, gaji, prev_month_balance) -> dict	10
2.4.2. Fungsi: predict_persona(features: dict) -> str	10
2.4.3. Enum Persona	11
2.5. RAG Service (rag_service.py)	12
2.5.1. System Prompt	12
2.5.2. Fungsi: build_weekly_context(...) -> str	12
2.5.3. Fungsi: build_monthly_context(...) -> str	13
2.5.4. Fungsi: call_llm(context: str, is_monthly: bool = False) -> str	13
2.5.5. Knowledge Base (knowledge/financial_kb.json)	13
3. Modul Scheduler (src/features/scheduler/)	15
3.1. Dependencies & Autentikasi (scheduler_dependencies.py)	15
3.1.1. verify_internal_key(x_internal_key: str = Header(...)) -> None	15
3.1.2. DI Factory Functions	15
3.2. Controller (scheduler_controller.py)	15

3.2.1. POST /scheduler/weekly	15
3.2.2. POST /scheduler/monthly	15
3.2.3. GET /scheduler/health	16
3.3. Use Case: Pipeline Mingguan (run_weekly_usecase.py)	17
3.3.1. Request Object	17
3.3.2. Alur Eksekusi	17
3.3.3. Domain Entity: WeeklyReport	18
3.4. Use Case: Pipeline Bulanan (run_monthly_usecase.py)	19
3.4.1. Request Object	19
3.4.2. Alur Eksekusi	19
3.4.3. Domain Entity: MonthlyReport	20
4. Konfigurasi	21
4.1. Environment Variables	21
4.2. Dependency Stack	21
5. Catatan Teknis Penting	22
5.1. PostgreSQL Native Enum	22
5.2. Rasio Wants vs Needs	22
5.3. Column Name Bridging (Autoencoder)	22
5.4. Error Handling	22

1. Gambaran Umum

FinSight adalah microservice FastAPI yang bertanggung jawab atas seluruh inferensi ML dan pembuatan laporan keuangan personal. Layanan ini berjalan secara internal (tidak terekspos ke publik) dan dikonsumsi oleh NestJS sebagai orchestrator utama.

Prinsip arsitektur:

- NestJS adalah *ground truth* untuk skema database — FastAPI tidak pernah membuat atau mengubah tabel maupun tipe enum PostgreSQL.
- FastAPI hanya membaca dan menulis data melalui SQLAlchemy async (`asyncpg`).
- Semua operasi ML berjalan sebagai *background task* (non-blocking).
- Model artifacts dimuat sekali saat startup menggunakan `@lru_cache`.

1.1. Struktur Direktori

```
src/
├── features/
│   └── scheduler/
│       ├── scheduler_controller.py # Route definitions (FastAPI Router)
│       ├── scheduler_dependencies.py # Auth + DI factories
│       ├── weekly/
│       │   └── schemas.py # Pydantic I/O schemas untuk weekly
│       ├── monthly/
│       │   └── schemas.py # Pydantic I/O schemas untuk monthly
│       └── usecase/
│           ├── run_weekly_usecase.py # Logika pipeline mingguan
│           └── run_monthly_usecase.py # Logika pipeline bulanan
└── ml/
    ├── model_loader.py # Lazy-load semua model artifacts
    ├── nlp_service.py # TF-IDF + klasifikasi P2P
    ├── autoencoder_service.py # Deteksi anomali transaksi
    ├── clustering_service.py # Feature engineering + prediksi persona
    ├── rag_service.py # Pembuat konteks + LLM coach
    └── knowledge/
        └── financial_kb.json # Knowledge base keuangan personal
```

1.2. Alur Data End-to-End

```
HTTP POST /scheduler/weekly
    |
    ▼
    scheduler_controller.py
    → generate job_id
    → BackgroundTasks.add_task()
    → return 202 Accepted
    |
    ▼ (async background)
    RunWeeklyUseCase.execute()
    → fetch transactions (last 7 days)
    → classify P2P via NLP
    → map main_category
    → preprocess for Autoencoder
    → detect_anomalies()
    → build_weekly_context()
    → call_llm() [OpenRouter API]
    → upsert WeeklyReport + DetectedAnomaly
```

2. Modul ML (`src/ml/`)

2.1. Model Loader (`model_loader.py`)

Bertanggung jawab untuk memuat semua model artifacts dari disk ke memori. Menggunakan `@lru_cache(maxsize=1)` sehingga setiap model hanya dimuat sekali selama lifetime proses.

2.1.1. Fungsi

Fungsi	Deskripsi	Return Type
<code>get_nlp_pipeline()</code>	Memuat TF-IDF vectorizer dan model klasifikasi dari path <code>settings.NLP_TOKENIZER_PATH</code> dan <code>settings.NLP_MODEL_PATH</code> .	<code>tuple[vectorizer, model]</code>
<code>get_autoencoder()</code>	Memuat Keras autoencoder dari <code>settings.AUTOENCODER_MODEL_PATH</code> tanpa kompilasi ulang (<code>compile=False</code>).	<code>tf.keras.Model</code>
<code>get_autoencoder_meta()</code>	Memuat dua file joblib: <code>preprocessing_meta.pkl</code> dan <code>model_meta.pkl</code> dari direktori yang sama dengan model Keras.	<code>tuple[dict, dict]</code>
<code>get_clustering_pipeline()</code>	Memuat empat artefak: <code>scaler_all.pkl</code> , <code>umap_all.pkl</code> , <code>kmeans_all_umap.pkl</code> , dan <code>label_map.json</code> untuk pipeline prediksi persona.	<code>tuple[scaler, umap, kmeans, dict]</code>
<code>preload_all_models()</code>	Dipanggil sekali saat startup aplikasi untuk <i>warm-up</i> semua model ke cache. Error pada satu model tidak menghentikan proses pemuatan model lain.	<code>None</code>

2.1.2. Model Artifacts

File	Lokasi	Deskripsi
<code>autoencoder.keras</code>	<code>model/autoencoder/</code>	Model Keras autoencoder untuk deteksi anomali.
<code>preprocessing_meta.pkl</code>	<code>model/autoencoder/</code>	Dict berisi: <code>ohe</code> , <code>scaler</code> , <code>kat_cap</code> , <code>z_stats</code> , <code>feature_cols</code> , <code>ohe_cols</code> .
<code>model_meta.pkl</code>	<code>model/autoencoder/</code>	Dict berisi: <code>threshold_user_kat</code> (per user+kategori) dan <code>threshold_kat</code> (per kategori).
<code>scaler_all.pkl</code>	<code>model/clustering/</code>	<code>StandardScaler</code> untuk 10 fitur perilaku bulanan.

umap_all.pkl	model/clustering/	UMAP dimensionality reduction model.
kmeans_all_umap.pkl	model/clustering/	KMeans model (3 cluster → persona).
label_map.json	model/clustering/	Mapping {cluster_id: persona_name}, e.g. {"0": "Tightwad", "1": "Unconflicted", "2": "Spendthrift"}.
tfidf_vectorizer.pkl	model/nlp/	TF-IDF vectorizer untuk teks transaksi P2P.
nlp_model.pkl	model/nlp/	Classifier (e.g. Logistic Regression) untuk prediksi sub-kategori dari teks.

2.2. NLP Service (nlp_service.py)

Modul ini menangani dua fungsi: reklasifikasi transaksi P2P menggunakan model NLP, dan pemetaan sub_category ke main_category (wants / needs / savings).

2.2.1. Konstanta

P2P_CATEGORY = "Transfer P2P" — nilai sentinel yang menandai transaksi P2P yang perlu diklasifikasikan ulang.

CATEGORY_MAP — dictionary yang memetakan nilai sub_category ke main_category. Mendukung dua format:

sub_category	main_category	Sumber
"Transportasi", "Tagihan & Utilitas", "Kesehatan & Perawatan Diri", "Groceries & Kebutuhan Pokok"	needs	Output model NLP (title case)
"Belanja Online & Fashion", "Produktivitas & Digital", "F&B dan Nongkrong", "Hiburan & Langganan", "Transfer P2P"	wants	Output model NLP (title case)
"Investasi & Finansial"	savings	Output model NLP (title case)
"sewa_kos", "belanja_dapur", "listrik", "internet", "bpjs_kesehatan", "makan", "transport"	needs	Nilai raw DB (snake_case)
"belanja_online", "hiburan", "makan_restoran", "gadget", "pakaian", "kafe", "langganan_streaming", "Makan & Minum"	wants	Nilai raw DB (snake_case)
"transfer_tabungan"	savings	Nilai raw DB (snake_case)
"Pendapatan Bulanan", "Pemasukan Tambahan"	None	Transaksi kredit — dikecualikan dari kalkulasi

2.2.2. Fungsi

_clean(text: str) -> str

Preprocessing teks untuk input NLP: lowercase, hapus karakter non-alphanumeric, normalisasi whitespace.

classify_p2p_transactions(df: DataFrame) -> DataFrame

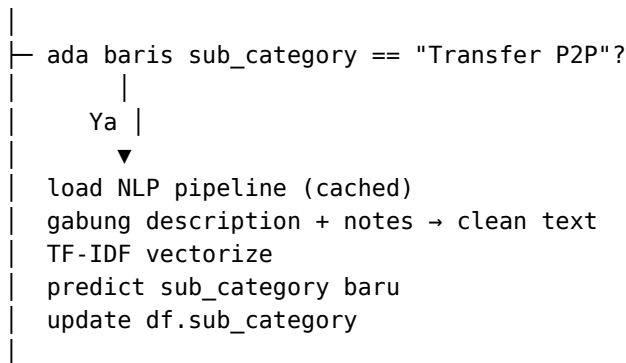
Reklasifikasi transaksi dengan `sub_category == "Transfer P2P"` menggunakan model NLP. Menggabungkan kolom `description` dan `notes` sebagai input teks. Mengembalikan DataFrame dengan `sub_category` diperbarui in-place. Jika model tidak tersedia, DataFrame dikembalikan tanpa perubahan.

map_main_category(df: DataFrame) -> DataFrame

Menambahkan kolom `main_category` ke DataFrame berdasarkan lookup `CATEGORY_MAP`. Nilai yang tidak ada di map → `None` (diabaikan dalam kalkulasi rasio).

2.2.3. Alur Proses

df masuk



map_main_category()

→ `df["main_category"] = df["sub_category"].map(CATEGORY_MAP)`

2.3. Autoencoder Service (`autoencoder_service.py`)

Modul deteksi anomali berbasis reconstruction error dari autoencoder neural network. Anomali adalah transaksi yang tidak sesuai dengan pola historis user di kategori tersebut.

2.3.1. Konstanta

- `MIN_Z = 3` — jumlah minimum transaksi historis user di kategori tertentu sebelum z-score dianggap valid.
- `_CONT_COLS = ["nominal", "hour_sin", "hour_cos", "nominal_z_user_kat"]` — nama kolom kontinu yang digunakan oleh model (sesuai nama saat training).

2.3.2. Fungsi: `preprocess_for_autoencoder(df, meta) -> np.ndarray`

Mempersiapkan DataFrame transaksi menjadi array numerik yang siap diinput ke autoencoder.

Langkah-langkah preprocessing:

1. **Rename kolom** — menyesuaikan nama kolom entity ke nama yang digunakan saat training:

Nama Entity (runtime)	Nama Model (training)
sub_category	kategori_detail
customer_id	id_user
amount	nominal

2. **Fitur waktu** — ekstrak hour dari `transaction_timestamp`, lalu encode siklik: `hour_sin = sin(2π × hour / 24)`, `hour_cos = cos(2π × hour / 24)`.
3. **Log transform nominal** — cap nominal berdasarkan `kat_cap` per kategori, lalu `log1p(nominal)`.
4. **One-hot encoding** — transform `kategori_detail` menggunakan OHE yang tersimpan di `meta["ohe"]`.
5. **Z-score per user-kategori** — merge dengan `z_stats` (mean dan std historis per `id_user × kategori_detail`). Hanya dihitung jika `z_count >= MIN_Z`. Hasil 0.0 jika data historis tidak cukup.
6. **Standard scaling** — transform 4 kolom kontinu menggunakan `meta["scaler"]`.
7. **Clip z-score** — nilai `nominal_z_user_kat_scaled` di-clip ke rentang `[-5, 5]`.
8. **Gabungkan fitur** — return array numpy shape `(n_rows, n_features)` dari semua kolom `_scaled`.

2.3.3. Fungsi: `detect_anomalies(df, X_scaled, model_meta, df_baseline, weekly_report_id)`

Menjalankan prediksi autoencoder dan menentukan anomali.

Algoritma:

1. Reconstruct input: `X_rec = autoencoder.predict(X_scaled)`.
2. Hitung per-row MAE: `mae = |X_scaled - X_rec|.mean(axis=1)`.
3. Tentukan threshold per transaksi: lookup di `threshold_user_kat` (per user+kategori), fallback ke `threshold_kat` (per kategori), fallback ke median semua threshold.
4. Hitung `mean_amount` historis per user×kategori dari `df_baseline`.

5. Suatu transaksi dinyatakan anomali jika: $mae > threshold$ AND $amount > mean_amount$.
Kondisi kedua memastikan hanya transaksi yang **lebih besar** dari biasanya yang diflagging.
6. Buat objek `DetectedAnomaly` untuk setiap baris anomali, lengkap dengan `anomaly_context`.

2.3.4. Fungsi: `_build_anomaly_context(row, df_baseline) -> str`

Menghasilkan teks narasi konteks anomali yang digunakan dalam laporan LLM.

Komponen konteks:

- Rasio nominal terhadap rata-rata historis user di kategori tersebut.
- Z-score (hanya ditampilkan jika $|z| \geq 2.0$).
- Deskripsi jam transaksi (dini hari / pagi / siang / sore / malam).

2.3.5. Domain Entity: `DetectedAnomaly`

Field	Tipe	Deskripsi
<code>transaction_id</code>	<code>str</code>	UUID transaksi yang terdeteksi anomali.
<code>customer_id</code>	<code>str</code>	ID nasabah.
<code>weekly_report_id</code>	<code>str</code>	UUID laporan mingguan yang menampung anomali ini.
<code>sub_category</code>	<code>str</code>	Kategori transaksi (setelah reklasifikasi NLP jika P2P).
<code>amount</code>	<code>int</code>	Nominal transaksi dalam Rupiah.
<code>mae</code>	<code>float</code>	Mean Absolute Error rekonstruksi autoencoder.
<code>threshold_val</code>	<code>float</code>	Threshold MAE untuk user+kategori ini.
<code>ratio</code>	<code>float</code>	$mae / (threshold_val + 1e-9)$ — severity score.
<code>anomaly_context</code>	<code>str</code>	Teks narasi konteks anomali untuk LLM.
<code>detected_at</code>	<code>Optional[datetime]</code>	Timestamp deteksi, diisi oleh database.

2.4. Clustering Service (`clustering_service.py`)

Modul yang menghitung 10 fitur perilaku keuangan bulanan dari data transaksi, lalu memprediksi persona nasabah menggunakan pipeline KMeans.

2.4.1. Fungsi: `compute_monthly_features(df_debit, df_all, gaji, prev_month_balance) -> dict`

Input:

- `df_debit` — DataFrame transaksi debit bulan ini (sudah terklasifikasi).
- `df_all` — DataFrame semua transaksi bulan ini (debit + kredit, untuk saldo).
- `gaji` — penghasilan bulanan nasabah dalam Rupiah.
- `prev_month_balance` — saldo akhir bulan sebelumnya.

Output — 10 fitur:

Fitur	Tipe	Definisi
<code>wants_ratio</code>	float	Total wants / total pengeluaran debit.
<code>fixed_costs_ratio</code>	float	Total needs / total pengeluaran debit.
<code>savings_rate</code>	float	$(\text{investasi} + \text{delta_saldo}) / \text{gaji}$.
<code>wants_frequency</code>	float	Jumlah transaksi wants / total transaksi debit.
<code>small_leaks_ratio</code>	float	Transaksi < Rp 30.000 / total transaksi debit.
<code>night_owl_spending</code>	float	Transaksi jam 22:00–04:00 / total transaksi debit.
<code>weekend_surge</code>	float	Rata-rata amount weekend / rata-rata amount weekday.
<code>early_month_depletion</code>	float	Total pengeluaran tanggal 1–5 / gaji.
<code>balance_volatility</code>	float	Std saldo harian / gaji.
<code>survival_mode_days</code>	int	Jumlah hari saldo < 15% dari gaji.

Catatan: Kolom `hour`, `day_of_week`, dan `day_of_month` bersifat opsional — jika tidak ada, fitur terkait di-default ke 0.

2.4.2. Fungsi: `predict_persona(features: dict) -> str`

Memprediksi persona nasabah dari 10 fitur. Return salah satu dari "Tightwad", "Unconflicted", atau "Spendthrift". Default ke "Unconflicted" jika model tidak tersedia atau prediksi gagal.

Pipeline:

1. Log-transform 4 fitur berikut untuk menstabilkan distribusi skewed: `wants_frequency`, `weekend_surge`, `balance_volatility`, `survival_mode_days`.
2. Buat array shape (1, 10) dari semua fitur sesuai urutan `ALL_FEATURES`.
3. `scaler.transform()` — `StandardScaler`.
4. `umap_model.transform()` — reduksi dimensi.
5. `kmeans.predict()` — prediksi cluster ID.
6. Lookup `label_map[cluster_id]` → string persona.

2.4.3. Enum Persona

Persona	Profil Singkat
Tightwad	Sangat hemat, wants ratio rendah ($< 15\%$), tabungan tinggi, namun berisiko mengorbankan kualitas hidup.
Unconflicted	Seimbang antara kebutuhan dan keinginan. Wants 20–35%, needs 40–55%, savings 15–25%. Pola paling sehat.
Spendthrift	Cenderung boros, wants ratio $> 40\%$, savings rate rendah atau negatif. Pemicu umum: impulse buying, FOMO.

2.5. RAG Service (rag_service.py)

Modul yang membangun konteks terstruktur dari data keuangan dan memanggil LLM eksternal (melalui OpenRouter) untuk menghasilkan laporan naratif personal.

2.5.1. System Prompt

SYSTEM_PROMPT_COACH:

Kamu adalah FinSight AI Coach – asisten keuangan personal yang berbicara seperti teman dekat yang jujur dan peduli.

Aturan WAJIB:

- Tulis dalam bentuk paragraf mengalir, bukan poin-poin atau daftar bernomor
- DILARANG menggunakan heading, subjudul, atau tanda bintang
- DILARANG menggunakan emoji atau emoticon dalam bentuk apapun
- Gunakan bahasa Indonesia yang hangat, natural, dan seperti percakapan
- Sertakan angka spesifik (nominal Rupiah, persentase) langsung dalam kalimat
- Tutup dengan 2-3 saran konkret dalam bentuk kalimat biasa, bukan poin

2.5.2. Fungsi: build_weekly_context(...) -> str

Membangun prompt konteks untuk laporan mingguan.

Parameter:

Parameter	Tipe	Deskripsi
user_id	str	ID nasabah.
user_name	str	Nama tampilan (saat ini sama dengan user_id).
persona	str	Persona aktif nasabah dari laporan bulanan terakhir.
gaji	float	Penghasilan bulanan (saat ini 0.0 — tidak digunakan di weekly).
saldo_terakhir	float	Running balance transaksi terakhir dalam periode.
wants_ratio	float	Proporsi wants dari total consumable spending.
needs_ratio	float	Proporsi needs dari total consumable spending.
wants_amount	float	Nominal wants dalam Rupiah.
needs_amount	float	Nominal needs dalam Rupiah.
total_pengeluaran	float	Total debit 7 hari terakhir.
anomali_list	list[dict]	Maks. 5 anomali teratas. Setiap dict: kategori, nominal, timestamp, context.
period_start	str	Tanggal mulai periode (YYYY-MM-DD).
period_end	str	Tanggal akhir periode (YYYY-MM-DD).

2.5.3. Fungsi: `build_monthly_context(...)` -> `str`

Membangun prompt konteks untuk laporan bulanan. Parameter tambahan dibanding weekly:

Parameter Tambahan	Tipe	Deskripsi
<code>persona_baru</code>	<code>str</code>	Persona hasil prediksi bulan ini.
<code>persona_lama</code>	<code>Optional[str]</code>	Persona bulan sebelumnya (ditampilkan jika berubah).
<code>savings_rate</code>	<code>float</code>	Tingkat tabungan bulan ini.
<code>savings_amount</code>	<code>float</code>	Nominal tabungan dalam Rupiah.
<code>behavioral_features</code>	<code>dict</code>	10 fitur perilaku dari <code>compute_monthly_features()</code> .
<code>target_month</code>	<code>str</code>	Bulan target format YYYY-MM.

2.5.4. Fungsi: `call_llm(context: str, is_monthly: bool = False)` -> `str`

Memanggil LLM melalui OpenRouter dengan format prompt yang sudah ditentukan.

Konfigurasi:

- Model: `settings.LLM_MODEL`
- API base URL: `settings.LLM_API_URL`
- Temperature: 0.7
- Max tokens: 600
- Extra body: `{"reasoning": {"enabled": True}}`

Format instruksi output — weekly:

Tepat 3 paragraf padat, tanpa heading, poin, atau emoji.

Paragraf 1: ringkasan pengeluaran – total, proporsi wants vs needs, kondisi saldo.

Paragraf 2: anomali atau pola yang perlu diperhatikan.

Paragraf 3: 2-3 saran konkret untuk minggu depan dalam kalimat biasa.

Tidak perlu basa-basi, sapaan panjang, atau pengulangan data.

Format instruksi output — monthly:

Tepat 3 paragraf padat, tanpa heading, poin, atau emoji.

Paragraf 1: ringkasan performa bulan ini – angka utama, persona, kondisi keuangan.

Paragraf 2: 2-3 pola perilaku paling menonjol dari data.

Paragraf 3: 2-3 saran konkret untuk bulan depan dalam kalimat biasa.

2.5.5. Knowledge Base (`knowledge/financial_kb.json`)

File JSON berisi artikel keuangan personal yang dapat digunakan sebagai RAG context. Setiap artikel memiliki struktur:

```
{
  "id": "rule_50_30_20",
  "title": "Aturan Anggaran 50/30/20 untuk Keuangan Sehat",
  "content": "...",
  "tags": ["budgeting", "50-30-20", "needs", "wants", "savings"]
}
```

Topik yang tersedia (13 artikel):

- `rule_50_30_20` — Aturan anggaran 50/30/20.

- `persona_tightwad`, `persona_unconflicted`, `persona_spendthrift` — Profil dan saran per persona.
- `anomali_transaksi` — Cara memahami dan merespons anomali.
- `survival_mode` — Tindakan saat saldo kritis.
- `early_month_depletion` — Efek paycheck / boros di awal bulan.
- `night_owl_spending` — Pengeluaran impulsif dini hari.
- `weekend_surge` — Lonjakan pengeluaran akhir pekan.
- `small_leaks` — Kebocoran kecil yang kumulatif.
- `savings_strategy` — Strategi menabung efektif.
- `balance_volatility` — Volatilitas saldo dan cash flow.
- `wants_control` — Teknik mengontrol pengeluaran wants.

3. Modul Scheduler (src/features/scheduler/)

3.1. Dependencies & Autentikasi (scheduler_dependencies.py)

Semua endpoint scheduler dilindungi oleh header autentikasi internal.

3.1.1. verify_internal_key(x_internal_key: str = Header(...)) -> None

Membandingkan nilai header X-Internal-Key dengan settings.INTERNAL_API_KEY. Mengembalikan HTTP 403 Forbidden jika tidak cocok.

Header yang diperlukan:

X-Internal-Key: <nilai dari env INTERNAL_API_KEY>

3.1.2. DI Factory Functions

```
def get_run_weekly_use_case() -> RunWeeklyUseCase
def get_run_monthly_use_case() -> RunMonthlyUseCase
```

Membuat instance use case baru per request (stateless). Diinjeksikan melalui Depends().

3.2. Controller (scheduler_controller.py)

Router FastAPI dengan prefix /scheduler dan tag "scheduler".

3.2.1. POST /scheduler/weekly

Response: 202 Accepted

Deskripsi: Memulai pipeline laporan mingguan sebagai background task. Mengembalikan respons segera tanpa menunggu pipeline selesai. Output pipeline hanya tersedia di server logs.

Request body (WeeklySchedulerRequest):

Field	Default	Deskripsi
customer_ids	[]	List UUID customer. Kosong = semua customer aktif di DB.
reference_date	null	Tanggal referensi. Null = hari ini. Pipeline memproses 7 hari sebelum tanggal ini.
dry_run	false	Jika true, pipeline berjalan penuh tapi tidak menulis ke DB.

Response body (WeeklyJobResponse):

```
{
  "job_id": "weekly-2026-05-25-abc123",
  "status": "queued",
  "customer_count": 0,
  "message": "Weekly scheduler job queued successfully"
}
```

3.2.2. POST /scheduler/monthly

Response: 202 Accepted

Deskripsi: Memulai pipeline laporan bulanan sebagai background task.

Request body (MonthlySchedulerRequest):

Field	Default	Deskripsi
customer_ids	[]	List UUID customer. Kosong = semua customer aktif.
target_month	null	Format YYYY-MM. Null = bulan lalu secara otomatis.
dry_run	false	Jika true, tidak menulis ke DB.

Validasi target_month: Regex `^\d{4}-\d{2}$`. Melempar 422 Unprocessable Entity jika format salah.

Response body (MonthlyJobResponse):

```
{
  "job_id": "monthly-2026-04-abc123",
  "status": "queued",
  "customer_count": 0,
  "target_month": "2026-04",
  "message": "Monthly scheduler job queued successfully"
}
```

3.2.3. GET /scheduler/health

Response: 200 OK

Deskripsi: Mengecek status pemuatan semua model ML. Memanggil setiap loader function dan melaporkan hasilnya.

Contoh response:

```
{
  "status": "ok",
  "models": {
    "nlp": "loaded",
    "autoencoder": "loaded",
    "clustering": "loaded"
  }
}
```

Jika model gagal dimuat, field akan berisi pesan error (maks. 60 karakter pertama).

3.3. Use Case: Pipeline Mingguan (`run_weekly_usecase.py`)

3.3.1. Request Object

```
@dataclass
class RunWeeklyRequest:
    job_id: str
    customer_ids: List[str] = []
    reference_date: Optional[date] = None # default: date.today()
    dry_run: bool = False
```

3.3.2. Alur Eksekusi

`execute(request):`

1. Hitung periode: `period_end = reference_date`, `period_start = period_end - 7 hari`.
2. Buka satu `AsyncSession` untuk seluruh job.
3. Ambil semua customer aktif jika `customer_ids` kosong.
4. Proses customer dalam chunk ukuran `CHUNK_SIZE = 50`.

`_process_chunk(...):`

1. Fetch transaksi debit 7 hari terakhir untuk chunk ini via `trx_repo.find_debit_last_7_days()`.
2. Fetch baseline historis untuk anomaly detection via `trx_repo.find_all_for_baseline()`.
3. Klasifikasi P2P dan mapping `main_category`.
4. Coba load preprocessing meta dan preprocess untuk autoencoder. Jika gagal, set `has_ae = False` dan lanjut tanpa anomaly detection.
5. Iterasi per customer dalam chunk, panggil `_generate_report()`.

`_generate_report(...):`

Langkah detail untuk satu customer:

Langkah	Deskripsi
1	Hitung <code>total_expenses</code> , <code>wants_nom</code> , <code>needs_nom</code> .
2	Hitung <code>wants_ratio</code> dan <code>needs_ratio</code> terhadap <code>consumable_total = wants_nom + needs_nom</code> (bukan total pengeluaran — mengecualikan savings).
3	Jalankan <code>detect_anomalies()</code> jika autoencoder tersedia. Ambil 3 anomali terparah untuk konteks LLM.
4	Ambil persona aktif dari <code>report_repo.get_latest_monthly_persona()</code> . Default "Unconflicted" jika belum ada.
5	Ambil saldo terakhir dari <code>running_balance</code> transaksi terakhir.
6	Bangun konteks dengan <code>build_weekly_context()</code> .
7	Panggil <code>call_llm()</code> . Jika gagal, gunakan konteks mentah sebagai fallback.
8	Buat objek <code>WeeklyReport</code> dan upsert ke DB (kecuali <code>dry_run</code>).
9	Simpan semua <code>DetectedAnomaly</code> ke DB, dengan <code>weekly_report_id</code> diisi setelah upsert.

3.3.3. Domain Entity: WeeklyReport

Field	Tipe	Deskripsi
id	Optional[str]	UUID, diisi oleh DB saat upsert.
customer_id	str	ID nasabah.
report_date	date	Tanggal akhir periode (= <code>period_end</code>).
period_start	date	Tanggal awal periode (7 hari sebelum <code>report_date</code>).
persona	str	Persona aktif nasabah.
wants_ratio	float	Proporsi wants dari consumable spending.
needs_ratio	float	Proporsi needs dari consumable spending.
total_expenses	int	Total pengeluaran debit 7 hari (Rupiah).
anomaly_count	int	Jumlah anomali terdeteksi.
report_text	str	Teks laporan 3 paragraf dari LLM.
generated_at	Optional[datetime]	Timestamp pembuatan, diisi oleh <code>func.now()</code> saat upsert.

3.4. Use Case: Pipeline Bulanan (`run_monthly_usecase.py`)

3.4.1. Request Object

```
@dataclass
class RunMonthlyRequest:
    job_id: str
    target_month: str # format YYYY-MM
    customer_ids: List[str] = []
    dry_run: bool = False
```

3.4.2. Alur Eksekusi

`execute(request):`

1. Parse `target_month` ke `year`, `month` → hitung `month_start` dan `month_end`.
2. Buka satu `AsyncSession`.
3. Ambil customer aktif jika list kosong.
4. Proses dalam chunk ukuran 50.

`_process_chunk(...):`

1. Fetch semua transaksi dalam bulan target via `trx_repo.find_debit_in_month()`.
2. Klasifikasi P2P dan mapping `main_category`.
3. Pisahkan `df_debit` (`type == "debit"`) dari `df_all` (semua).
4. Iterasi per customer.

`_generate_report(...):`

Langkah	Deskripsi
1	Ambil <code>persona</code> sebelumnya dari <code>report_repo.get_latest_monthly_persona()</code> .
2	Ambil gaji dari <code>customer_repo.get_monthly_income()</code> — query kolom <code>monthly_income</code> di tabel <code>customer</code> .
3	Hitung <code>prev_month_balance</code> (saldo pertama bulan ini = saldo akhir bulan lalu) dan <code>saldo_akhir</code> .
4	Hitung 10 fitur via <code>compute_monthly_features()</code> .
5	Prediksi <code>persona_baru</code> via <code>predict_persona()</code> .
6	Hitung <code>wants_nom</code> , <code>needs_nom</code> , rasio terhadap <code>consumable_total</code> .
7	Hitung <code>savings_amount = investasi_nom + max(delta_saldo, 0)</code> . Investasi dari sub_category “Investasi & Finansial”, <code>delta_saldo = saldo_akhir - saldo_awal</code> .
8	Bangun konteks bulanan dengan <code>build_monthly_context()</code> .
9	Panggil <code>call_llm(is_monthly=True)</code> . Fallback ke konteks mentah jika gagal.
10	Upsert <code>MonthlyReport</code> ke DB.
11	Update <code>base_persona</code> di tabel <code>customer</code> via <code>customer_repo.update_base_persona()</code> .

3.4.3. Domain Entity: MonthlyReport

Field	Tipe	Deskripsi
id	Optional[str]	UUID.
customer_id	str	ID nasabah.
target_month	str	Bulan laporan format YYYY-MM.
persona	PersonaEnum	Persona baru hasil prediksi bulan ini.
prev_persona	Optional[PersonaEnum]	Persona bulan sebelumnya.
savings_rate	float	Tingkat tabungan dari <code>compute_monthly_features()</code> .
wants_ratio	float	Proporsi wants dari consumable.
needs_ratio	float	Proporsi needs dari consumable.
wants_amount	int	Nominal wants dalam Rupiah.
needs_amount	int	Nominal needs dalam Rupiah.
savings_amount	int	Investasi + kenaikan saldo bersih.
behavioral_features	dict	10 fitur perilaku (disimpan sebagai JSONB).
report_text	str	Teks laporan 3 paragraf dari LLM.
generated_at	Optional[datetime]	Timestamp, diisi DB.

4. Konfigurasi

4.1. Environment Variables

Variable	Digunakan di	Deskripsi
INTERNAL_API_KEY	scheduler_dependencies.py	Key autentikasi header X-Internal-Key.
LLM_API_URL	rag_service.py	Base URL OpenRouter (atau LLM provider lain yang OpenAI-compatible).
LLM_API_KEY	rag_service.py	API key untuk LLM provider.
LLM_MODEL	rag_service.py	Nama model LLM, e.g. "openai/gpt-4o-mini".
AUTOENCODER_MODEL_PATH	model_loader.py	Path ke file autoencoder.keras.
KMEANS_MODEL_PATH	model_loader.py	Path ke file kmeans_all_umap.pkl.
KMEANS_LABEL_MAP_PATH	model_loader.py	Path ke file label_map.json.
NLP_TOKENIZER_PATH	model_loader.py	Path ke file tfidf_vectorizer.pkl.
NLP_MODEL_PATH	model_loader.py	Path ke file nlp_model.pkl.

4.2. Dependency Stack

Library	Versi	Kegunaan
fastapi	≥0.100	HTTP framework, Router, BackgroundTasks, Depends.
sqlalchemy	≥2.0	Async ORM, text(), func.now().
asyncpg	—	PostgreSQL async driver.
pydantic	v2	Request/response validation, field_validator.
pandas	—	Manipulasi DataFrame transaksi.
numpy	—	Array operations untuk preprocessing.
tensorflow / keras	—	Load dan inferensi model autoencoder.
joblib	—	Load sklearn artifacts (scaler, KMeans, UMAP).
umap-learn	—	Dimensionality reduction untuk clustering.
openai	—	AsyncOpenAI client untuk call LLM.

5. Catatan Teknis Penting

5.1. PostgreSQL Native Enum

NestJS/TypeORM membuat tipe enum native di PostgreSQL. SQLAlchemy **tidak boleh** mencoba membuat ulang tipe ini. Semua kolom enum di SQLAlchemy harus didefinisikan dengan:

```
from sqlalchemy.dialects.postgresql import ENUM

_trx_type_enum = ENUM(
    'debit', 'credit',
    name='transactions_transaction_type_enum',
    create_type=False # <-- kritis: jangan buat ulang
)
```

Nama tipe enum harus persis sesuai yang ada di PostgreSQL (verifiable via \dT di psql).

5.2. Rasio Wants vs Needs

$\text{wants_ratio} + \text{needs_ratio} = 1.0$ dijamin karena keduanya dihitung terhadap `consumable_total`:

```
consumable_total = wants_nom + needs_nom # savings dikecualikan
wants_ratio = wants_nom / consumable_total
needs_ratio = needs_nom / consumable_total
```

Transaksi dengan `main_category == "savings"` atau `None` tidak dimasukkan dalam denominasi.

5.3. Column Name Bridging (Autoencoder)

Model autoencoder dilatih dengan nama kolom yang berbeda dari nama entity domain. Rename dilakukan di awal `preprocess_for_autoencoder()`, sebelum operasi model apapun:

Entity (runtime)	Model artifact (training)
sub_category	kategori_detail
customer_id	id_user
amount	nominal

5.4. Error Handling

Semua error di dalam loop customer bersifat non-fatal — satu customer yang gagal tidak menghentikan pemrosesan customer lain. Error fatal (DB tidak bisa diakses, session gagal dibuka) akan menghentikan seluruh pipeline dan tercatat di logs.

Skenario	Penanganan
Model tidak tersedia (file tidak ada)	<code>preload_all_models()</code> log error, pipeline lanjut tanpa model tersebut.
AE preprocessing gagal	<code>has_ae = False</code> , laporan dibuat tanpa deteksi anomali.
LLM call gagal (timeout, 5xx)	Gunakan raw context string sebagai <code>report_text</code> dengan prefix "[LLM unavailable]".
Customer tidak ada transaksi	Log warning, skip customer, lanjut ke berikutnya.

Clustering gagal	<code>predict_persona()</code> return "Unconflicted" sebagai default.
------------------	---